

Informal Design Guidelines for Relation Schemas

we discuss four informal guidelines that may be used as measures to determine the quality of relation schema design:

- Making sure that the semantics of the attributes is clear in the schema
- Reducing the redundant information in tuples
- Reducing the NULL values in tuples
- Disallowing the possibility of generating spurious tuples

Imparting Clear Semantics to Attributes in Relations

- The semantics of a relation refers to its meaning resulting from the interpretation of attribute values in a tuple.

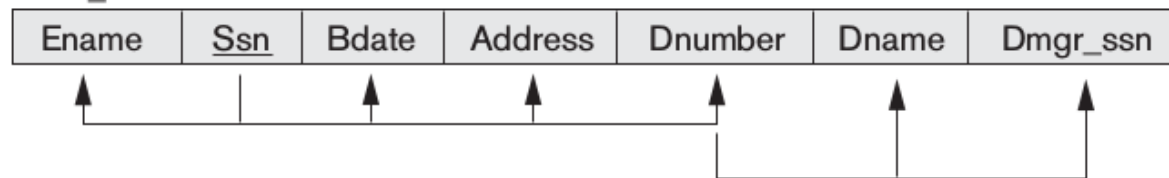
Guideline 1

- Design a relation schema so that it is easy to explain its meaning.
- Do not combine attributes from multiple entity types and relationship types into a single relation.

- **Examples of Violating Guideline 1.** The relation schemas in Figures below also have clear semantics.
- Although there is nothing wrong logically with these two relations, they violate Guideline 1 by mixing attributes from distinct real-world entities: EMP_DEPT mixes attributes of employees and departments, and EMP_PROJ mixes attributes of employees and projects and the WORKS_ON relationship.

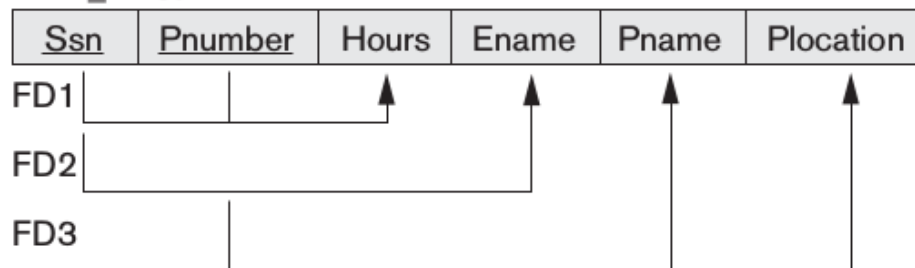
(a)

EMP_DEPT



(b)

EMP_PROJ



Redundant Information in Tuples and Update Anomalies

- Grouping attributes into relation schemas has a significant effect on storage space.
- For example, compare the space used by the two base relations EMPLOYEE and DEPARTMENT with that for an EMP_DEPT base relation, which is the result of applying the NATURAL JOIN operation to EMPLOYEE and DEPARTMENT.
- In EMP_DEPT, the attribute values pertaining to a particular department (Dnumber, Dname, Dmgr_ssn) are repeated for every employee who works for that department.

- Storing natural joins of base relations leads to an additional problem referred to as update anomalies.
- These can be classified into **insertion anomalies**, **deletion anomalies**, and **modification anomalies**.
- **Insertion Anomalies**
 - To insert a new employee tuple into EMP_DEPT, we must include either the attribute values for the department that the employee works for, or NULLs.
 - It is difficult to insert a new department that has no employees as yet in the EMP_DEPT relation. The only way to do this is to place NULL values in the attributes for employee. This violates the entity integrity for EMP_DEPT because Ssn is its primary key.

- **Deletion Anomalies** If we delete from EMP_DEPT an employee tuple that happens to represent the last employee working for a particular department, the information concerning that department is lost from the database.
- **Modification Anomalies** In EMP_DEPT , if we change the value of one of the attributes of a particular department—say, the manager of department 5—we must update the tuples of all employees who work in that department; otherwise, the database will become inconsistent.

Guideline 2

- Design the base relation schemas so that no insertion, deletion, or modification anomalies are present in the relations. The second guideline is consistent with and, in a way, a restatement of the first guideline.

NULL Values in Tuples

- If many of the attributes do not apply to all tuples in the relation, we end up with many NULLs in those tuples.
- This can waste space at the storage level and may also lead to problems with understanding the meaning of the attributes.
- Another problem with NULLs is how to account for them when aggregate operations such as COUNT or SUM are applied.
- SELECT and JOIN operations involve comparisons; if NULL values are present, the results may become unpredictable.

Guideline 3

- As far as possible, avoid placing attributes in a base relation whose values may frequently be NULL.
- If NULLs are unavoidable, make sure that they apply in exceptional cases only and do not apply to a majority of tuples in the relation.

Generation of Spurious Tuples

- Consider the two relation schemas EMP_LOCS and EMP_PROJ1 in Figure below, which can be used instead of the single EMP_PROJ relation.
- If we attempt a NATURAL JOIN operation on EMP_PROJ1 and EMP_LOCS, the result produces many more tuples than the original set of tuples in EMP_PROJ.
- Additional tuples that were not in EMP_PROJ are called spurious tuples because they represent spurious information that is not valid.
- This is because in this case Plocation is the attribute that relates EMP_LOCS and EMP_PROJ1, and Plocation is neither a primary key nor a foreign key in either EMP_LOCS or EMP_PROJ1.

(a)

EMP_LOCS

<u>Ename</u>	<u>Plocation</u>
--------------	------------------

P.K.

EMP_PROJ1

<u>Ssn</u>	<u>Pnumber</u>	Hours	Pname	Plocation
------------	----------------	-------	-------	-----------

P.K.

Guideline 4

- Design relation schemas so that they can be joined with equality conditions on attributes that are appropriately related (primary key, foreign key) pairs in a way that guarantees that no spurious tuples are generated.
- Avoid relations that contain matching attributes that are not (foreign key, primary key) combinations because joining on such attributes may produce spurious tuples.

Functional Dependencies

- **Definition.** A functional dependency, denoted by $X \rightarrow Y$, between two sets of attributes X and Y that are subsets of R (relation schema) specifies a constraint on the possible tuples that can form a relation state r of R .
- The constraint is that, for any two tuples t_1 and t_2 in r that have $t_1[X] = t_2[X]$, they must also have $t_1[Y] = t_2[Y]$.
- We say that there is a functional dependency from X to Y , or that Y is functionally dependent on X .
- If $X \rightarrow Y$ in R , this does not say whether or not $Y \rightarrow X$ in R .

- Consider the relation schema EMP_PROJ; we know that the following functional dependencies should hold:
 - a. $Ssn \rightarrow Ename$
 - b. $Pnumber \rightarrow \{ Pname, Plocation \}$
 - c. $\{ Ssn, Pnumber \} \rightarrow Hours$
- These functional dependencies specify that
 - a. the value of an employee's Social Security number (Ssn) uniquely determines the employee name (Ename),
 - b. the value of a project's number (Pnumber) uniquely determines the project name (Pname) and location (Plocation)
 - c. a combination of Ssn and Pnumber values uniquely determines the number of hours the employee currently works on the project per week (Hours).

Normal Forms Based on Primary Keys

- We assume that a set of functional dependencies is given for each relation, and that each relation has a designated primary key; this information combined with the tests (conditions) for normal forms drives the normalization process for relational schema design.

Normalization of Relations

- The normalization process, as first proposed by Codd, takes a relation schema through a series of tests to certify whether it satisfies a certain normal form.
- Normalization of data can be considered a process of analyzing the given relation schemas based on their FDs and primary keys to achieve the desirable properties of (1) minimizing redundancy and (2) minimizing the insertion, deletion, and update anomalies to make the design have successively better quality.

- **Definition.** The normal form of a relation refers to the highest normal form condition that it meets, and hence indicates the degree to which it has been normalized.
- The process of normalization through decomposition must also confirm the existence of additional properties that the relational schemas, taken together, should possess.
- These would include two properties:
- The **nonadditive join or lossless join property**, which guarantees that the spurious tuple generation problem discussed, does not occur with respect to the relation schemas created after decomposition.
- The **dependency preservation property**, which ensures that each functional dependency is represented in some individual relation resulting after decomposition.

Definitions of Keys and Attributes Participating in Keys

- **Definition.** A **superkey** of a relation schema $R = \{A_1, A_2, \dots, A_n\}$ is a set of attributes $S \subseteq R$ with the property that no two tuples t_1 and t_2 in any legal relation state r of R will have $t_1[S] = t_2[S]$.
- A **key** K is a superkey with the additional property that removal of any attribute from K will cause K not to be a superkey any more.
- The difference between a key and a superkey is that a key has to be minimal; that is, if we have a key $K = \{A_1, A_2, \dots, A_k\}$ of R , then $K - \{A_i\}$ is not a key of R for any $A_i, 1 \leq i \leq k$.
- If a relation schema has more than one key, each is called a **candidate key**.
- **Definition.** An attribute of relation schema R is called a **prime attribute** of R if it is a member of some candidate key of R .

First Normal Form

- First normal form was defined to disallow multivalued attributes, composite attributes, and their combinations.
- It states that the domain of an attribute must include only atomic (simple, indivisible) values and that the value of any attribute in a tuple must be a single value from the domain of that attribute.
- Consider the DEPARTMENT relation schema, whose primary key is Dnumber, and suppose that we extend it by including the Dlocations attribute as shown in (a).
- We assume that each department can have a number of locations. As we can see, this is not in 1NF because Dlocations is not an atomic attribute.

(a)

DEPARTMENT

Dname	<u>Dnumber</u>	Dmgr_ssn	Dlocations
			

(b)

DEPARTMENT

Dname	<u>Dnumber</u>	Dmgr_ssn	Dlocations
Research	5	333445555	{Bellaire, Sugarland, Houston}
Administration	4	987654321	{Stafford}
Headquarters	1	888665555	{Houston}

(c)

DEPARTMENT

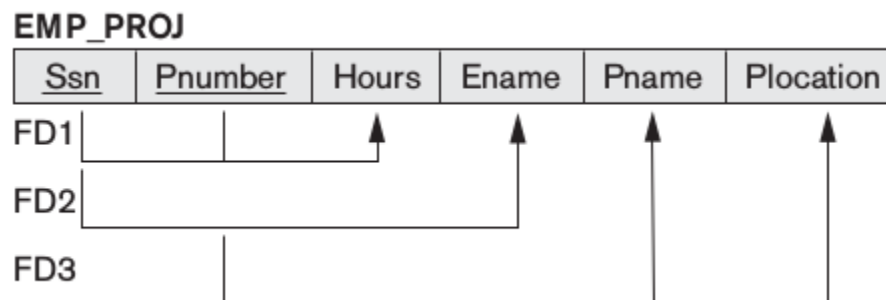
Dname	<u>Dnumber</u>	Dmgr_ssn	<u>Dlocation</u>
Research	5	333445555	Bellaire
Research	5	333445555	Sugarland
Research	5	333445555	Houston
Administration	4	987654321	Stafford
Headquarters	1	888665555	Houston

There are three main techniques to achieve first normal form for such a relation:

1. Remove the attribute Dlocations that violates 1NF and place it in a separate relation DEPT_LOCATIONS along with the primary key Dnumber of DEPARTMENT. The primary key of this relation is the combination {Dnumber, Dlocation}.
2. Expand the key so that there will be a separate tuple in the original DEPARTMENT relation for each location of a DEPARTMENT, as shown in Figure(c). In this case, the primary key becomes the combination { Dnumber, Dlocation }. This solution has the disadvantage of introducing redundancy in the relation.
3. If a maximum number of values is known for the attribute—for example, if it is known that at most three locations can exist for a department—replace the Dlocations attribute by three atomic attributes: Dlocation1, Dlocation2, and Dlocation3. This solution has the disadvantage of introducing NULL values if most departments have fewer than three locations.

Second Normal Form

- Second normal form (2NF) is based on the concept of full functional dependency.
- A functional dependency $X \rightarrow Y$ is a full functional dependency if removal of any attribute A from X means that the dependency does not hold any more;
- That is, for any attribute $A \in X$, $(X - \{A\})$ does not functionally determine Y . A functional dependency $X \rightarrow Y$ is a partial dependency if some attribute $A \in X$ can be removed from X and the dependency still holds;
- In Figure below, $\{ \text{Ssn}, \text{Pnumber} \} \rightarrow \text{Hours}$ is a full dependency (neither $\text{Ssn} \rightarrow \text{Hours}$ nor $\text{Pnumber} \rightarrow \text{Hours}$ holds). However, the dependency $\{ \text{Ssn}, \text{Pnumber} \} \rightarrow \text{Ename}$ is partial because $\text{Ssn} \rightarrow \text{Ename}$ holds.

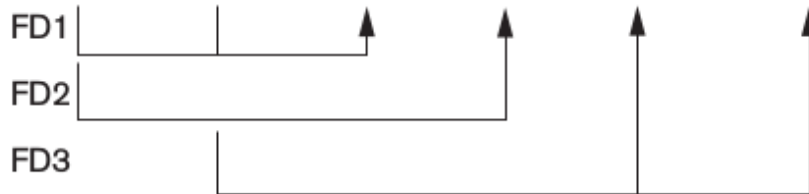


- **Definition.** A relation schema R is in 2NF if every nonprime attribute A in R is fully functionally dependent on the primary key of R .
- If the primary key contains a single attribute, the test need not be applied at all.
- The EMP_PROJ relation in Figure above is in 1NF but is not in 2NF. The nonprime attribute Ename violates 2NF because of FD2, as do the nonprime attributes Pname and Plocation because of FD3.
- If a relation schema is not in 2NF, it can be 2NF normalized into a number of 2NF relations in which nonprime attributes are associated only with the part of the primary key on which they are fully functionally dependent.

(a)

EMP_PROJ

<u>Ssn</u>	<u>Pnumber</u>	Hours	Ename	Pname	Plocation
------------	----------------	-------	-------	-------	-----------



2NF Normalization

EP1

<u>Ssn</u>	<u>Pnumber</u>	Hours
------------	----------------	-------



EP2

<u>Ssn</u>	Ename
------------	-------



EP3

<u>Pnumber</u>	Pname	Plocation
----------------	-------	-----------

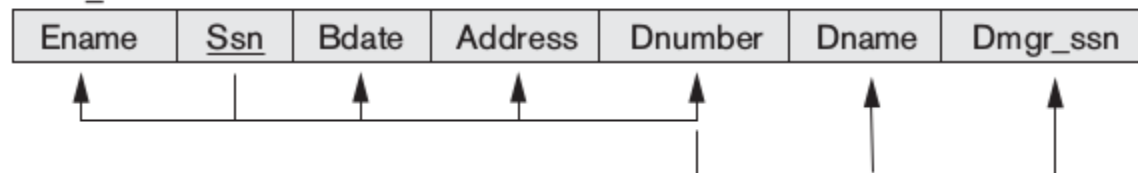


Third Normal Form

- Third normal form (3NF) is based on the concept of transitive dependency.
- A functional dependency $X \rightarrow Y$ in a relation schema R is a transitive dependency if there exists a set of attributes Z in R that is neither a candidate key nor a subset of any key of R , and both $X \rightarrow Z$ and $Z \rightarrow Y$ hold.
- The dependency $Ssn \rightarrow Dmgr_ssn$ is transitive through $Dnumber$ in EMP_DEPT in Figure below.

(a)

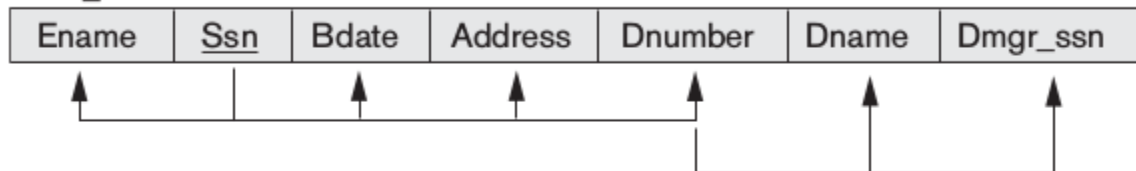
EMP_DEPT



- **Definition.** a relation schema R is in 3NF if it satisfies 2NF and no nonprime attribute of R is transitively dependent on the primary key.
- The relation schema EMP_DEPT in Figure below is in 2NF, since no partial dependencies on a key exist.
- However, EMP_DEPT is not in 3NF because of the transitive dependency of Dmgr_ssn (and also Dname) on Ssn via Dnumber.
- A NATURAL JOIN operation on ED1 and ED2 will recover the original relation EMP_DEPT without generating spurious tuples.

(b)

EMP_DEPT



3NF Normalization

ED1



ED2



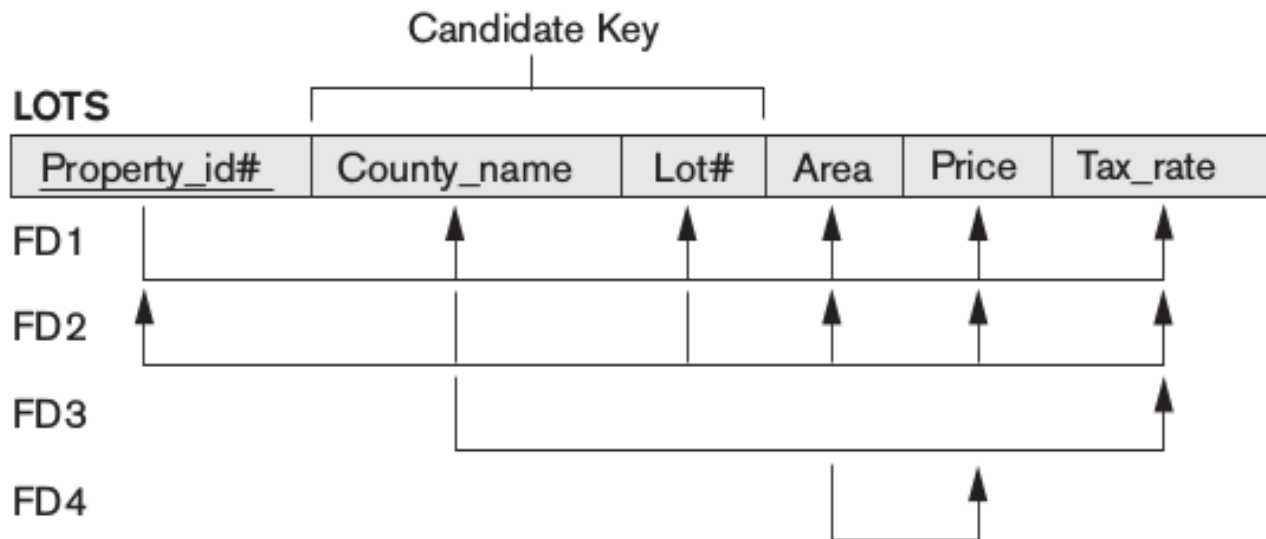
General Definitions of Second and Third Normal Forms

- The normalization procedure described so far is useful for a given database where primary keys have already been defined.
- These definitions, however, do not take other candidate keys of a relation, if any, into account.
- Partial and full functional dependencies and transitive dependencies will now be considered with respect to all candidate keys of a relation.

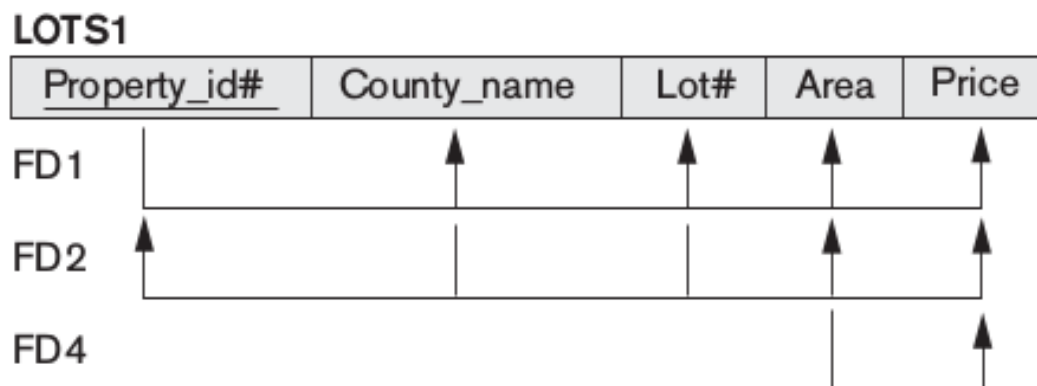
General Definition of Second Normal Form

- **Definition.** A relation schema R is in second normal form (2NF) if every nonprime attribute A in R is not partially dependent on any key of R .
- The test for 2NF involves testing for functional dependencies whose left-hand side attributes are part of the key.
- If the key contains a single attribute, the test need not be applied at all.
- Consider the relation schema $LOTS$ shown in Figure below, which describes parcels of land for sale in various counties of a state.

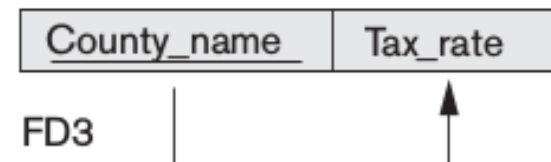
(a)



(b)



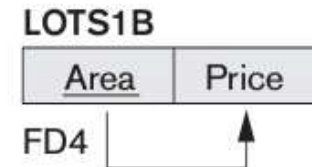
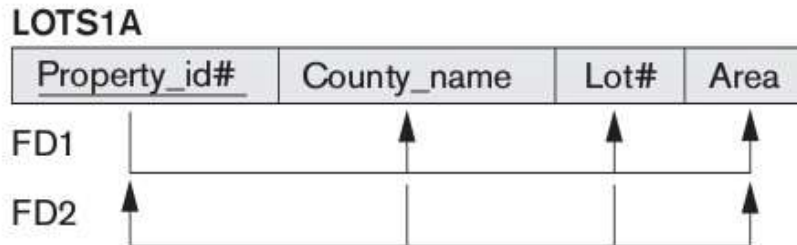
LOTS2



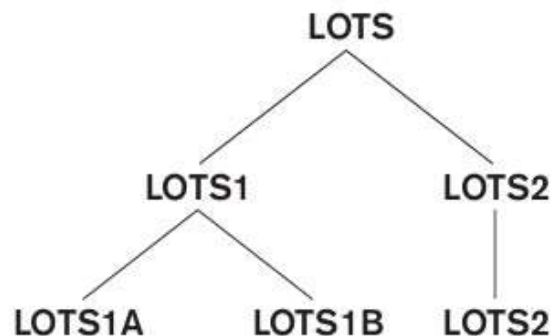
General Definition of Third Normal Form

- **Definition.** A relation schema R is in third normal form (3NF) if, whenever a nontrivial functional dependency $X \rightarrow A$ holds in R , either (a) X is a superkey of R , or (b) A is a prime attribute of R .
- According to this definition, $LOTS2$ is in 3NF. However, $FD4$ in $LOTS1$ violates 3NF because $Area$ is not a superkey and $Price$ is not a prime attribute in $LOTS1$.

(c)



(d)



1NF

2NF

3NF

Boyce-Codd Normal Form

- We can see the need for a stronger normal form than 3NF by going back to the LOTS relation schema.
- Suppose that we have thousands of lots in the relation but the lots are from only two counties: DeKalb and Fulton.
- Suppose also that lot sizes in DeKalb County are only 0.5, 0.6, 0.7, 0.8, 0.9, and 1.0 acres, whereas lot sizes in Fulton County are restricted to 1.1, 1.2, ..., 1.9, and 2.0 acres.
- In such a situation we would have the additional functional dependency FD5: **Area**→**County_name**.
- If we add this to the other dependencies, the relation schema LOTS1A still is in 3NF because County_name is a prime attribute.

(a)

LOTS1A

<u>Property_id#</u>	County_name	Lot#	Area
---------------------	-------------	------	------

FD1

FD2

FD5

BCNF Normalization

LOTS1AX

<u>Property_id#</u>	Area	Lot#
---------------------	------	------

LOTS1AY

<u>Area</u>	County_name
-------------	-------------

Definition. A relation schema R is in BCNF if whenever a nontrivial functional dependency $X \rightarrow A$ holds in R , then X is a superkey of R .